Dossier Técnico

Integración de Supabase en Aplicaciones Multiplataforma (Ionic / Angular)

Módulo Profesional: Desarrollo de Interfaces (Código 0488)
Ciclo Formativo: Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM2)

Alineación Curricular Comunidad de Madrid:

- **RA 1:** Confecciona interfaces gráficas de usuario utilizando herramientas visuales y librerías de componentes (patrones arquitectónicos MVVM/Clean Architecture, sincronización reactiva de estado interfaz-backend).
  - *CE 1.a:* Se han identificado los componentes de la arquitectura de la interfaz.
  - *CE 1.c:* Se han establecido los mecanismos de enlace entre los datos del modelo y la vista.
- **RA 3:** Crea componentes visuales personalizados evaluando y gestionando eventos.
  - *CE 3.d:* Se han gestionado eventos de autenticación y flujos de sesión persistente en los componentes de interfaz.

1. Base Teórica Técnica: Arquitectura y Capacidades de Supabase

Supabase se define como una alternativa de código abierto a Firebase construida sobre una arquitectura modular de servicios alrededor de **PostgreSQL 15+**. A diferencia de las soluciones BaaS basadas en bases de datos orientadas a documentos (NoSQL con esquemas dinámicos no normalizados), Supabase expone un motor relacional transaccional (ACID) completo que traslada la lógica de seguridad y validación al propio motor de datos.

Servicios Núcleo del Ecosistema

1. **PostgREST (Capa RESTful declarativa):**
   - Inspecciona el esquema public de PostgreSQL y genera automáticamente una API RESTful sin código intermedio.
   - Modela operaciones CRUD y transformaciones de datos directamente a sentencias SQL optimizadas, serializando los resultados en JSON nativo desde el propio motor (json_agg).
   - Soporta filtrado relacional avanzado, paginación por cursores o desplazamientos y embebido de relaciones uno-a-muchos o muchos-a-muchos sin incurrir en el problema $N + 1$.

2. **GoTrue (Motor de Autenticación y Autorización):**
   - Microservicio escrito en Go que administra el ciclo de vida de usuarios, autenticación basada en contraseñas, proveedores OAuth 2.0 externos (GitHub, Google, Apple) y Magic Links.
   - Emite tokens **JWT (JSON Web Tokens)** firmados con algoritmo HMAC-SHA256 (HS256) o asimétricos (RS256), que transportan el ID del usuario (sub), correo y metadatos (app_metadata, user_metadata)

3. **Row-Level Security (RLS) y Control de Acceso Basado en Roles (RBAC):**
   - La seguridad no reside en la API, sino en el motor de base de datos. PostgreSQL evalúa las políticas RLS por cada fila afectada en cada consulta (SELECT, INSERT, UPDATE, DELETE).
   - El JWT recibido por Kong se decodifica y transfiere a variables de sesión de PostgreSQL:
     - auth.uid(): Retorna el UUID del usuario autenticado extraído del token.
     - auth.jwt(): Devuelve el payload completo del JWT para inspección de claims personalizados.
   - La gestión de roles se implementa mediante esquemas con tablas pivote (user_roles) o inyectando un claim app_metadata.role validable mediante funciones de base de datos ejecutadas en el contexto de la sesión.

4. **Realtime Engine (Elixir / Phoenix):**
   - Escucha la replicación lógica de PostgreSQL (vía protocolo wal2json o pgoutput) y emite eventos WebSocket (INSERT, UPDATE, DELETE) en tiempo real hacia los clientes suscritos a canales específicos.
5. **Storage:**
   - Abstracción compatible con el API S3 para almacenar archivos multimedia (avatares, documentos), cuyas carpetas y objetos se rigen de manera idéntica por políticas RLS en la tabla storage.objects.

2. Implementación Paso a Paso: Integración en Ionic / Angular (Standalone)

2.1. Definición del Esquema y Seguridad en PostgreSQL

Ejecutar en el **SQL Editor** del panel de Supabase:

```
SQL
-- 1. Definición del ENUM de roles del sistema
CREATE TYPE public.app_role AS ENUM ('admin', 'developer', 'viewer');

-- 2. Tabla de perfiles vinculada a la tabla interna auth.users
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT NOT NULL,
  full_name TEXT,
  role public.app_role NOT NULL DEFAULT 'viewer',
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL
);
```

```
-- 3. Habilitar Row Level Security (RLS)
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

-- 4. Función de utilidad para evaluar roles en las políticas RLS (Security Definer para aislar permisos)
CREATE OR REPLACE FUNCTION public.has_role(required_role public.app_role)
RETURNS BOOLEAN
LANGUAGE sql
STABLE
SECURITY DEFINER
AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.profiles
    WHERE id = auth.uid()
      AND role = required_role
  );
$$;

-- 5. Políticas RLS:
-- Lectura: Los usuarios leen su propio perfil; los admins leen todos los perfiles
CREATE POLICY "Profiles are readable by owner or admin"
ON public.profiles
FOR SELECT
USING (
  auth.uid() = id OR public.has_role('admin')
);

-- Actualización: Solo administradores pueden modificar el rol de un perfil
CREATE POLICY "Only admins can update roles"
ON public.profiles
FOR UPDATE
USING (public.has_role('admin'))
WITH CHECK (public.has_role('admin'));

-- 6. Trigger para auto-crear perfil al registrarse en GoTrue
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  INSERT INTO public.profiles (id, email, full_name, role)
```

```
  VALUES (
    NEW.id,
    NEW.email,
    NEW.raw_user_meta_data->>'full_name',
    COALESCE((NEW.raw_app_meta_data->>'role')::public.app_role, 'viewer')
  );
  RETURN NEW;
END;
$$;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

2.2. Configuración del Cliente en Ionic/Angular

Instalación de la librería oficial de Supabase:

```
Bash
npm install @supabase/supabase-js
```

Definición de variables en src/environments/environment.ts:

```
TypeScript
export const environment = {
 production: false,
 supabase: {
  url: 'https://TU-PROYECTO.supabase.co',
  anonKey: 'TU-ANON-KEY-PUBLICA'
 }
};
```

2.3. Servicio de Infraestructura de Autenticación y Estado: supabase.service.ts

```
TypeScript
import { Injectable, signal, computed, WritableSignal } from '@angular/core';
import {
  createClient,
  SupabaseClient,
  User,
  Session,
  AuthChangeEvent
} from '@supabase/supabase-js';
import { environment } from 'src/environments/environment';

export interface UserProfile {
  id: string;
  email: string;
  full_name: string | null;
  role: 'admin' | 'developer' | 'viewer';
}

@Injectable({
  providedIn: 'root'
})
export class SupabaseService {
  private supabase: SupabaseClient;

  // Reactividad moderna mediante Angular Signals
  private currentUserSignal: WritableSignal<User | null> = signal<User | null>(null);
  private currentProfileSignal: WritableSignal<UserProfile | null> = signal<UserProfile |
null>(null);

  public readonly user = computed(() => this.currentUserSignal());
  public readonly profile = computed(() => this.currentProfileSignal());
  public readonly isAuthenticated = computed(() => !!this.currentUserSignal());
  public readonly isAdmin = computed(() => this.currentProfileSignal()?.role === 'admin');

  constructor() {
    this.supabase = createClient(
      environment.supabase.url,
      environment.supabase.anonKey,
      {
        auth: {
          persistSession: true,
```

```
        autoRefreshToken: true,
        detectSessionInUrl: true
      }
    }
  );

  this.initSessionListener();
}

get client(): SupabaseClient {
  return this.supabase;
}

private async initSessionListener(): Promise<void> {
  // 1. Carga inicial de sesión existente
  const { data: { session } } = await this.supabase.auth.getSession();
  await this.processSession(session);

  // 2. Suscripción a cambios de estado de autenticación (GoTrue)
  this.supabase.auth.onAuthStateChange(
    async (event: AuthChangeEvent, session: Session | null) => {
      await this.processSession(session);
    }
  );
}

private async processSession(session: Session | null): Promise<void> {
  if (session?.user) {
    this.currentUserSignal.set(session.user);
    await this.fetchUserProfile(session.user.id);
  } else {
    this.currentUserSignal.set(null);
    this.currentProfileSignal.set(null);
  }
}

private async fetchUserProfile(userId: string): Promise<void> {
  const { data, error } = await this.supabase
    .from('profiles')
    .select('id, email, full_name, role')
    .eq('id', userId)
    .single();
```

```
    if (error) {
      console.error('Error recuperando perfil:', error.message);
      this.currentProfileSignal.set(null);
      return;
    }

    this.currentProfileSignal.set(data as UserProfile);
  }

  async signIn(email: string, password: string): Promise<void> {
    const { error } = await this.supabase.auth.signInWithPassword({ email, password });
    if (error) throw error;
  }

  async signOut(): Promise<void> {
    const { error } = await this.supabase.auth.signOut();
    if (error) throw error;
  }
}
```

2.4. Control de Acceso a Rutas: auth.guard.ts

```
TypeScript
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { SupabaseService } from './supabase.service';

export const roleGuard = (expectedRole: 'admin' | 'developer' | 'viewer'): CanActivateFn => {
  return async () => {
    const supabaseService = inject(SupabaseService);
    const router = inject(Router);

    // Esperar a que la señal resuelva si se accede mediante refresh directo
    const profile = supabaseService.profile();

    if (profile && profile.role === expectedRole) {
      return true;
    }

    // Redirección si no cumple con el nivel de privilegios
    return router.createUrlTree(['/unauthorized']);
  };
};
```